

Hebbian Learning Rule — AND Gate Implementation

Source: [GeeksforGeeks — Hebbian Learning Rule with Implementation of AND Gate](#)

□ Background

The **Hebbian Learning Rule** (Donald O. Hebb, 1949) is one of the oldest and simplest learning rules in neural networks. It is summarised as:

"Neurons that fire together, wire together."

Weight Update Formula

$$w_{\text{new}} = w_{\text{old}} + x_i \cdot y$$

$$b_{\text{new}} = b_{\text{old}} + y$$

Architecture

- **Single-layer** network: one input layer + one output layer
 - Activation function: **Bipolar** (values in {-1, +1})
 - No hidden layers
-

Contents

1. Baseline implementation (zero initialisation — as per the article)
2. Alternative weight initialisation methods:
 - Random Uniform
 - Random Normal (Gaussian)
 - Xavier / Glorot
 - He Initialisation
 - Constant (small positive)

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from prettytable import PrettyTable

np.random.seed(42)
```

1 · Dataset — AND Gate (Bipolar)

Using the **bipolar** representation: False → -1, True → +1.

x_1	x_2	Bias	Target (y)
-1	-1	1	-1
-1	1	1	-1
1	-1	1	-1
1	1	1	1

```
# Inputs: [x1, x2, bias] | Targets
X = np.array([
    [-1, -1, 1],
    [-1, 1, 1],
    [1, -1, 1],
    [1, 1, 1]
])

y = np.array([-1, -1, -1, 1])

print('Input matrix X (x1, x2, bias):')
print(X)
print('\nTarget vector y:', y)

Input matrix X (x1, x2, bias):
[[-1 -1  1]
 [-1  1  1]
 [ 1 -1  1]
 [ 1  1  1]]

Target vector y: [-1 -1 -1  1]
```

2 · Core Hebbian Learning Function

```
def hebbian_train(X, y, w_init):
    """
    Train a single-layer network using the Hebbian Learning Rule.

    Parameters
    -----
    X      : ndarray (n_samples, n_features) – input matrix (last col
= bias)
    y      : ndarray (n_samples,)           – target outputs
    w_init : ndarray (n_features,)          – initial weights [w1,
w2, ..., b]

    Returns
```

```

-----
w          : final weight vector
history    : list of weight vectors after each sample
"""
w = w_init.copy().astype(float)
history = [w.copy()]

for i, (xi, yi) in enumerate(zip(X, y)):
    w = w + xi * yi          #  $\Delta w = x \cdot y$  (Hebbian rule)
    history.append(w.copy())

return w, history

def hebbian_predict(X, w):
    """Compute net input and predict output sign."""
    net = X @ w
    return net, np.sign(net).astype(int)

def evaluate(X, y, w, label=''):
    """Print a prediction table and return accuracy."""
    net, pred = hebbian_predict(X, w)
    table = PrettyTable()
    table.field_names = ['x1', 'x2', 'Net Input', 'Predicted',
                        'Target', 'Correct?']
    correct = 0
    for i in range(len(y)):
        ok = '✓' if pred[i] == y[i] else 'x'
        correct += (pred[i] == y[i])
        table.add_row([X[i,0], X[i,1], round(net[i],4), pred[i], y[i],
ok])
    acc = correct / len(y) * 100
    print(f'=== {label} ===' if label else '')
    print(table)
    print(f'Accuracy: {correct}/{len(y)} = {acc:.1f}%\n')
    return acc

```

3 · Method 1 — Zero Initialisation (Original Article)

All weights and bias start at 0, exactly as described in the GFG article.

```

print('— Zero Initialisation —')
w0 = np.zeros(3)          # [w1, w2, bias_weight]
print(f'Initial weights: {w0}\n')

w_zero, hist_zero = hebbian_train(X, y, w0)

```

```
print('\n--- Testing the network ---')
acc_zero = evaluate(X, y, w_zero, label='Zero Init')

— Zero Initialisation —
Initial weights: [0. 0. 0.]

--- Testing the network ---
=== Zero Init ===
```

x1	x2	Net Input	Predicted	Target	Correct?
-1	-1	-6.0	-1	-1	✓
-1	1	-2.0	-1	-1	✓
1	-1	-2.0	-1	-1	✓
1	1	2.0	1	1	✓

```
Accuracy: 4/4 = 100.0%
```

4 · Decision Boundary (Zero Init)

From the final weights [2, 2, -2]:

$$2x_1 + 2x_2 - 2 = 0 \implies x_2 = -x_1 + 1$$

```
def plot_decision_boundary(w, title='Decision Boundary', ax=None):
    if ax is None:
        fig, ax = plt.subplots(figsize=(5, 5))

    # Data points
    colors = ['red' if t == -1 else 'green' for t in y]
    labels = ['AND=0 (bipolar -1)', 'AND=1 (bipolar +1)']
    for xi, yi, c in zip(X, y, colors):
        ax.scatter(xi[0], xi[1], color=c, s=120, zorder=5)
        ax.annotate(f'({xi[0]}, {xi[1]}) \ ny={yi}',
                    (xi[0], xi[1]), textcoords='offset points',
                    xytext=(8, 5), fontsize=8)

    # Decision boundary: w[0]*x1 + w[1]*x2 + w[2]*1 = 0
    if abs(w[1]) > 1e-6:
        x1_vals = np.linspace(-2, 2, 100)
        x2_vals = -(w[0] * x1_vals + w[2]) / w[1]
        ax.plot(x1_vals, x2_vals, 'b--', linewidth=2, label='Decision
boundary')

    ax.set_xlim(-2, 2); ax.set_ylim(-2, 2)
```

```

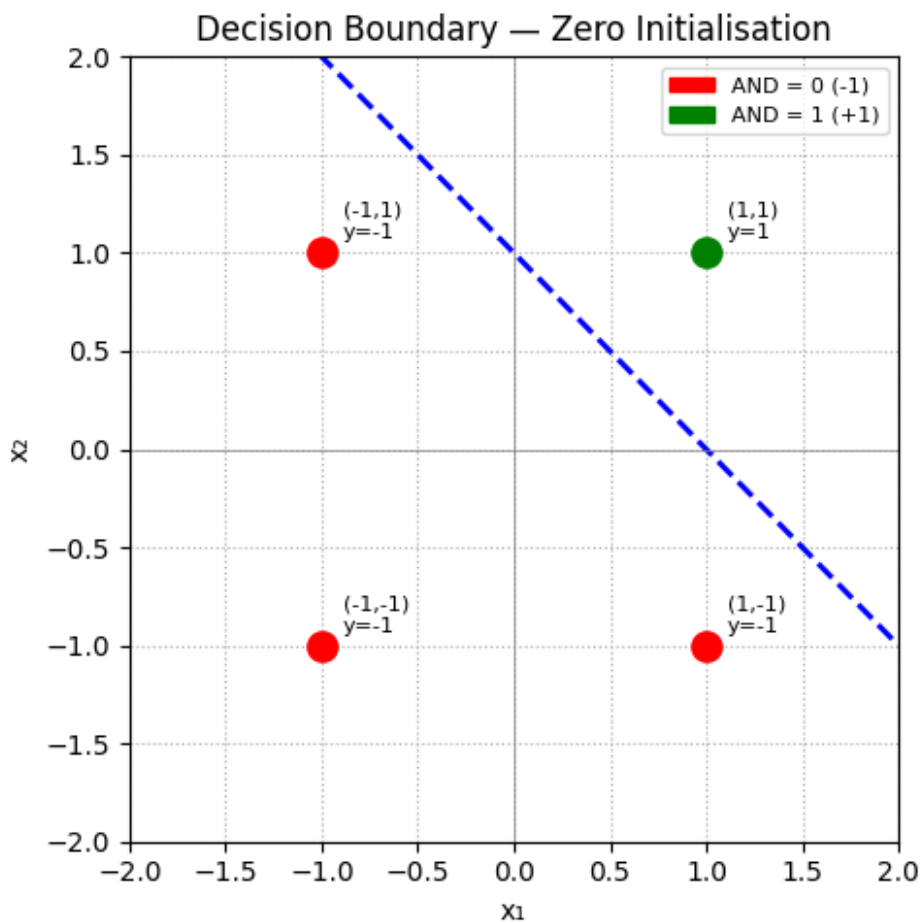
ax.axhline(0, color='grey', linewidth=0.5)
ax.axvline(0, color='grey', linewidth=0.5)
ax.set_xlabel('x1'); ax.set_ylabel('x2')
ax.set_title(title)
red_p = mpatches.Patch(color='red', label='AND = 0 (-1)')
grn_p = mpatches.Patch(color='green', label='AND = 1 (+1)')
ax.legend(handles=[red_p, grn_p], loc='upper right', fontsize=8)
ax.grid(True, linestyle=':')

```

```

plot_decision_boundary(w_zero, title='Decision Boundary – Zero
Initialisation')
plt.tight_layout()
plt.show()

```



5 • Alternative Weight Initialisation Methods

We now explore **five** different initialisation strategies and compare how they affect the Hebbian update trajectory and final decision boundaries.

Method	Description
Zero	All weights = 0 (baseline)
Random Uniform	Uniform in $[-0.5, 0.5]$
Random Normal	Gaussian, $\mu=0$, $\sigma=0.1$
Xavier / Glorot	Uniform in $[-1/\sqrt{n}, 1/\sqrt{n}]$
He	Gaussian, $\sigma=\sqrt{2/n}$
Constant	All weights = 0.1

```

n_features = X.shape[1] # 3 (x1, x2, bias)

def init_zero(n):
    """All zeros – standard Hebbian starting point."""
    return np.zeros(n)

def init_random_uniform(n, low=-0.5, high=0.5):
    """Uniform random in [low, high]."""
    return np.random.uniform(low, high, n)

def init_random_normal(n, mean=0.0, std=0.1):
    """Gaussian / Normal distribution."""
    return np.random.normal(mean, std, n)

def init_xavier(n):
    """Xavier / Glorot uniform: limit = 1/sqrt(n)."""
    limit = 1.0 / np.sqrt(n)
    return np.random.uniform(-limit, limit, n)

def init_he(n):
    """He initialisation: std = sqrt(2/n)."""
    std = np.sqrt(2.0 / n)
    return np.random.normal(0.0, std, n)

def init_constant(n, val=0.1):
    """All weights set to a small constant."""
    return np.full(n, val)

# — Gather all methods

init_methods = {
    'Zero': init_zero(n_features),
    'Random Uniform': init_random_uniform(n_features),
    'Random Normal': init_random_normal(n_features),
    'Xavier': init_xavier(n_features),
    'He': init_he(n_features),
    'Constant (0.1)': init_constant(n_features, 0.1),
}

print('Initial weight vectors for each method:')

```

```
tbl = PrettyTable(['Method', 'w1', 'w2', 'bias_w'])
for name, w in init_methods.items():
    tbl.add_row([name] + [f'{v:.4f}' for v in w])
print(tbl)
```

Initial weight vectors for each method:

Method	w1	w2	bias_w
Zero	0.0000	0.0000	0.0000
Random Uniform	-0.1255	0.4507	0.2320
Random Normal	-0.1112	0.0319	0.0279
Xavier	0.2403	-0.5536	0.5426
He	0.8251	-0.3833	0.4430
Constant (0.1)	0.1000	0.1000	0.1000

6 · Run Hebbian Learning for Every Initialisation

```
results = {} # name → {'w_init', 'w_final', 'history', 'accuracy'}
```

```
for name, w_init in init_methods.items():
    print(f' {name} Initialisation')
    print(f' Initial weights: {np.round(w_init, 4)}')
    w_final, history = hebbian_train(X, y, w_init)
    acc = evaluate(X, y, w_final, label=name)
    results[name] = {
        'w_init' : w_init.copy(),
        'w_final' : w_final,
        'history' : history,
        'accuracy': acc
    }
```

Zero Initialisation

Initial weights: [0. 0. 0.]

=== Zero ===

x1	x2	Net Input	Predicted	Target	Correct?
-1	-1	-6.0	-1	-1	✓
-1	1	-2.0	-1	-1	✓
1	-1	-2.0	-1	-1	✓
1	1	2.0	1	1	✓

Accuracy: 4/4 = 100.0%

Random Uniform Initialisation

Initial weights: [-0.1255 0.4507 0.232]

=== Random Uniform ===

x1	x2	Net Input	Predicted	Target	Correct?
-1	-1	-6.0933	-1	-1	✓
-1	1	-1.1918	-1	-1	✓
1	-1	-2.3442	-1	-1	✓
1	1	2.5572	1	1	✓

Accuracy: 4/4 = 100.0%

Random Normal Initialisation

Initial weights: [-0.1112 0.0319 0.0279]

=== Random Normal ===

x1	x2	Net Input	Predicted	Target	Correct?
-1	-1	-5.8928	-1	-1	✓
-1	1	-1.829	-1	-1	✓
1	-1	-2.1152	-1	-1	✓
1	1	1.9486	1	1	✓

Accuracy: 4/4 = 100.0%

Xavier Initialisation

Initial weights: [0.2403 -0.5536 0.5426]

=== Xavier ===

x1	x2	Net Input	Predicted	Target	Correct?
-1	-1	-5.1441	-1	-1	✓
-1	1	-2.2512	-1	-1	✓
1	-1	-0.6636	-1	-1	✓
1	1	2.2293	1	1	✓

Accuracy: 4/4 = 100.0%

He Initialisation

Initial weights: [0.8251 -0.3833 0.443]

=== He ===

x1	x2	Net Input	Predicted	Target	Correct?
-1	-1	-5.9988	-1	-1	✓
-1	1	-2.7654	-1	-1	✓
1	-1	-0.3486	-1	-1	✓
1	1	2.8848	1	1	✓

Accuracy: 4/4 = 100.0%

Constant (0.1) Initialisation
Initial weights: [0.1 0.1 0.1]

=== Constant (0.1) ===

x1	x2	Net Input	Predicted	Target	Correct?
-1	-1	-6.1	-1	-1	✓
-1	1	-1.9	-1	-1	✓
1	-1	-1.9	-1	-1	✓
1	1	2.3	1	1	✓

Accuracy: 4/4 = 100.0%